

Accenture Custom Software Engineer Interview Prep

Comprehensive Questions & Answers Bank (Python, SQL, DSA, ML, LLMs, AWS & PySpark)

Candidate: Kishan Sinha | Profile: AI/ML & Data Engineer

Section 1: HR Questions & Answers

Q1: Tell me about yourself.

Answer:

I am an AI/ML and Data Engineer with a strong educational background in technology, holding a Bachelor of Engineering in Electronics and Communication and a Post Graduate Diploma in Artificial Intelligence (PG-DAI) from CDAC. Over my academic and project-building career, I have focused on constructing end-to-end Machine Learning pipelines, advanced GenAI/RAG systems, and scalable big data processing workflows.

Specifically, I have built:

1. A production-grade **Retrieval-Augmented Generation (RAG) chatbot** using LangChain, Groq API, and FAISS with custom caching and query condensation.
2. An unsupervised **Time-Series Anomaly Detection System (SentinelGuard)** utilizing LSTM and GRU Autoencoders on market index returns.
3. A **Retail Sales Forecasting pipeline** comparing Random Forest and XGBoost with rigorous feature engineering.

I am highly proficient in Python, SQL, PySpark, FastAPI, Docker, and AWS, and I specialize in integrating machine learning models with robust backend APIs.

Q2: Why do you want to join Accenture?

Answer:

Accenture is an industry leader in enterprise digital transformation, cloud migrations, and artificial intelligence solutions. As a Custom Software Engineering candidate with specialized skills in AI/ML and big data, I want to join Accenture because you work with Global 2000 clients to solve real-world problems at a massive scale. Accenture's focus on scaling generative AI capabilities and integrating cloud architectures (AWS/Azure) perfectly matches my hands-on experience with FastAPI, PySpark, LangChain, and Docker. I am excited by the collaborative, forward-thinking tech culture and the opportunity to build high-performance software modules under your engineering framework.

Q3: Why should we hire you?

Answer:

You should hire me because I offer a unique combination of **Software Engineering, Data Engineering, and Machine Learning capabilities**:

- **Software Development**: Proficient in Python, writing clean, modular code, and building high-throughput asynchronous REST APIs using FastAPI.
- **Data Engineering**: Solid foundation in SQL databases (PostgreSQL/MySQL), PySpark, and Delta Lake Medallion pipelines for cleansing and processing high-volume datasets.
- **Machine Learning & GenAI**: Practical experience building and evaluating forecasting models (XGBoost, Random Forest), deep autoencoders (LSTM/GRU), and advanced RAG architectures using FAISS and LangChain.

Reflected in my CDAC training, I package my software using Docker, integrate CI/CD workflows, and run pipelines on AWS. I can step in, understand existing pipelines, and contribute immediately.

Q4: Tell me about your strengths.

Answer:

My greatest strengths are **problem-solving adaptability, end-to-end technical execution, and attention to performance optimization**.

- *Adaptability*: Transitioning from ECE to AI/ML and Data Engineering required intense self-driven learning and academic excellence at CDAC (achieving 81.47%).
- *End-to-End Execution*: I don't just write scripts; I build data ingestion pipelines, train models, develop FastAPI backends, compile Streamlit frontends, and deploy using Docker.
- *Optimization*: I actively look for bottlenecks—such as building an on-disk embedding cache in my RAG project to bypass redundant API calls and speed up document re-indexing from seconds to milliseconds.

Q5: Tell me about your weaknesses.

Answer:

My main weakness is that I sometimes get overly focused on micro-optimizations. During the development of my RAG chatbot, I spent excessive time trying to squeeze milliseconds out of the embedding ingestion pipeline before the core conversational memory was fully tested. I realized this was delaying my Minimum Viable Product (MVP). To manage this, I now apply strict time-boxing to my tasks: I focus first on delivering a fully functional baseline system, and only allocate specific, limited periods to optimization after the primary deliverables are stable.

Q6: Describe a challenging project and how you handled it.

Answer:

The most challenging project was my **RAG Q&A System**. The primary challenges were **hallucinations** and **conversational context loss** in multi-turn chats.

- **Context Loss:** Follow-up queries like "When did he join?" lacked context, causing similarity search in FAISS to fail. I solved this by adding a **Question Condensation Step** using LangChain to rewrite historical turns into a standalone query.
- **Hallucinations:** The LLM would make up facts if information was missing. I solved this with a two-layer guardrail: strict instructions in the system prompt to refuse queries lacking context, and a **post-inference Python filter** that detected the refusal string and wiped out the source citations block to protect citation integrity.

Q7: How do you handle pressure and tight deadlines?

Answer:

I handle pressure by remaining structured, prioritizing tasks, and communicating transparently. If a deadline is tight:

1. I break down the goal into smaller, atomic tasks.
2. I identify high-priority dependencies (e.g., database schema stability must precede API routing).
3. I build a working prototype first to de-risk key integrations.
4. I communicate progress early with team members if any unforeseen technical blocker arises.

Q8: Where do you see yourself in five years?

Answer:

In five years, I see myself growing into a **Technical Architect** or **Principal Engineer** specializing in AI/ML and Big Data solutions. I want to lead teams in designing scalable cloud data lakehouses, streaming pipelines (Kafka/Spark Streaming), and production-grade Generative AI agents. I look forward to obtaining advanced cloud certifications (AWS Solution Architect / Databricks Associate) and taking on leadership responsibilities to mentor junior developers at Accenture.

Q9: Are you willing to relocate?

Answer:

Yes, I am completely willing to relocate to any of Accenture's key technology hub locations. I am excited about the opportunity to work in a collaborative, in-office environment and engage directly with the engineering teams.

Q10: Do you have any questions for us?

Answer:

Yes, I have two questions:

1. For someone in the Custom Software Developer role, what are the primary cloud platforms (AWS, Azure, or GCP) and database technologies that your team is currently focusing on for upcoming enterprise projects?

2. How does Accenture integrate generative AI and automated tools (like GitHub Copilot) into the daily software development lifecycle of your engineering teams?

\n# Section 2: Python Q&A

Q11: Explain the difference between a List and a Tuple. When would you use which?

Answer:

- **List:** A mutable, dynamic array. Created using square brackets `[]`. It supports operations like `append()`, `extend()`, and `remove()`. It is slower and occupies more memory because Python over-allocates memory to accommodate future appends.
- **Tuple:** An immutable, fixed-size sequence. Created using parentheses `()`. Its values cannot be modified, added, or removed after creation. It is faster and consumes less memory.
- **Use Case:** Use lists for collections of data that will change dynamically at runtime (e.g., appending transaction logs). Use tuples for fixed, structured records (e.g., `(latitude, longitude)`) and as dictionary keys, which requires hashability.

Q12: Compare Python Dictionaries and Sets.

Answer:

- **Dictionary:** A collection of key-value pairs (`{key: value}`). Keys must be unique and hashable (immutable objects like strings, numbers, or tuples). Values can be duplicates and of any type.
- **Set:** An unordered collection of unique elements (`{val1, val2}`). It is essentially a dictionary where keys have no associated values.
- **Commonality:** Both use internal hash tables for lookup, insertion, and deletion, giving them an average time complexity of $O(1)$.

Q13: What is the difference between Mutable and Immutable objects in Python?

Answer:

- **Mutable:** Objects whose state or content can be changed in-place after creation without changing their memory address. Examples: `list`, `dict`, `set`, custom classes.
- **Immutable:** Objects whose value cannot be changed. Modifying them creates a new object in memory with a different id. Examples: `int`, `float`, `str`, `tuple`, `frozenset`.
- **Implication:** Modifying mutable objects in function calls affects the caller directly (passed by sharing), while immutable objects remain unaffected because re-assignment changes the local binding.

Q14: What is a Generator and how does it save memory?

Answer:

A generator is a special type of iterator defined by a function that uses the `yield` keyword instead of `return`.

- **Mechanism:** Instead of calculating and loading an entire list of values into RAM at once, a generator yields one value at a time on-demand (lazy evaluation). When **yield** is called, the function's state, variable values, and instruction pointer are suspended and saved. They resume on the next call to **__next__()** or in a **for** loop.
- **Memory Savings:** Generating 1,000,000 items in a list consumes megabytes of memory. A generator producing the same items consumes only a few bytes, representing just the size of the generator object state.

Q15: What is an Iterator in Python? How does it differ from an Iterable?

Answer:

- **Iterable:** An object that can return its elements one by one. It implements the **__iter__()** method, which returns an iterator. Examples: lists, strings, dictionaries.
- **Iterator:** An object that represents a data stream. It implements the **__next__()** method (which returns the next element or raises **StopIteration** when empty) and the **__iter__()** method (which returns **self**).
- **Relationship:** Every iterator is an iterable, but not every iterable is an iterator. For example, a list is iterable, but you cannot call **next(my_list)** directly until you convert it: **my_iterator = iter(my_list)**.

Q16: What are Decorators and how do you write a custom one?

Answer:

A decorator is a design pattern that wraps a function, modifying or extending its behavior without changing its source code. It takes a function as an argument and returns a new wrapper function.

```
def my_decorator(func):
    def wrapper(*args, **kwargs):
        print("Before function execution.")
        result = func(*args, **kwargs)
        print("After function execution.")
        return result
    return wrapper

@my_decorator
def say_hello(name):
    print(f"Hello, {name}")
```

Q17: Explain the usage of ***args** and ****kwargs** in functions.

Answer:

- ***args:** Captures positional arguments that exceed the formal parameter list, packaging them as a **tuple**. It allows a function to accept a variable number of positional inputs.
- ****kwargs:** Captures keyword arguments that exceed the formal parameter list, packaging them as a **dictionary**.

- **Usage:** Extremely useful in writing wrappers, decorators, and subclassing where you want to pass arguments directly to the parent class initializer: `super().__init__(*args, **kwargs)`.

Q18: What is an anonymous (Lambda) function? Show an example.

Answer:

A lambda function is a small, anonymous function defined with the **lambda** keyword. It can take any number of arguments but can only contain a single expression, which is evaluated and returned automatically.

```
# Lambda to compute square
square = lambda x: x ** 2
print(square(5)) # Output: 25

# Sorting tuples by second element using lambda
pairs = [(1, 'one'), (2, 'two'), (3, 'three')]
pairs.sort(key=lambda x: x[1])
```

Q19: Explain List Comprehensions with an example.

Answer:

List comprehension is a concise syntax for creating lists from existing iterables. It replaces verbose **for** loops and improves readability.

- **Syntax:** `[expression for item in iterable if condition]`

```
# Traditional loop
squares = []
for x in range(10):
    if x % 2 == 0:
        squares.append(x ** 2)

# List comprehension
squares_comp = [x ** 2 for x in range(10) if x % 2 == 0]
```

Q20: How does Exception Handling work in Python?

Answer:

Python handles runtime errors using **try**, **except**, **else**, and **finally** blocks:

- **try:** The block of code containing potential exceptions is executed.
- **except:** Catch and handle specific exceptions (e.g., **ValueError**, **KeyError**).
- **else:** Executes if no exception was raised in the **try** block.

- **finally**: Always executes, regardless of whether an exception occurred, making it ideal for clean-up tasks (like closing files or database connections).

Q21: What is the best practice for File Handling in Python?

Answer:

The best practice is to use the context manager **with** statement. It guarantees that file descriptors are automatically closed when execution leaves the block, even if an exception occurs.

```
# Best Practice
with open('data.txt', 'r', encoding='utf-8') as file:
    content = file.read()
    # file is automatically closed here
```

Q22: What are the core Object-Oriented Programming (OOP) concepts?

Answer:

1. **Inheritance**: Creating new classes (child) from existing ones (parent) to reuse code.
2. **Polymorphism**: The ability of different classes to implement methods with the same name but different behaviors.
3. **Encapsulation**: Bundling state (attributes) and behavior (methods) into a class and restricting direct access to components (e.g., private variables prefix `__`).
4. **Abstraction**: Hiding internal implementation details and exposing only essential interfaces using abstract base classes (**abc** module).

Q23: Explain Inheritance in Python.

Answer:

Inheritance allows a subclass to inherit attributes and methods from a superclass. Python supports single, multiple, multilevel, hierarchical, and hybrid inheritance.

```
class Parent:
    def greet(self):
        return "Hello from Parent"

class Child(Parent):
    def play(self):
        return "Playing..."

c = Child()
print(c.greet()) # Inherited method
```

Q24: What is Polymorphism? How is it implemented?

Answer:

Polymorphism means "many forms". In Python, it is implemented through:

- **Method Overriding:** Subclass redefines a method from the parent class.
- **Duck Typing:** Python doesn't check types; it checks for the existence of methods. "If it walks like a duck and quacks like a duck, it's a duck."

```
class Dog:
    def speak(self): return "Woof!"

class Cat:
    def speak(self): return "Meow!"

def make_animal_speak(animal):
    print(animal.speak()) # Polymorphic behavior
```

Q25: Explain Encapsulation. How are private variables declared in Python?

Answer:

Encapsulation wraps data and methods into a single class and controls access:

- **Public:** Accessible anywhere (e.g., **self.name**).
- **Protected:** Single underscore prefix (**self._salary**). Indicates it shouldn't be accessed outside the class (by convention).
- **Private:** Double underscore prefix (**self.__bank_account**). Python triggers **name mangling**, renaming it internally to **__ClassName__bank_account** to prevent direct external access.

Q26: What is the difference between Multithreading and Multiprocessing?

Answer:

- **Multithreading:** Threads run in the same memory space. Ideal for I/O-bound tasks (network requests, DB operations). However, due to Python's Global Interpreter Lock (GIL), only one thread executes bytecode at a time, rendering it ineffective for CPU-bound multi-core processing.
- **Multiprocessing:** Processes run in separate memory spaces. Ideal for CPU-bound tasks (machine learning training, matrix math). It bypasses the GIL by spawning distinct OS processes, utilizing multiple CPU cores.

Q27: What is the Global Interpreter Lock (GIL)?

Answer:

The GIL is a mutex (lock) used by CPython (the default Python implementation) to ensure that only one thread executes Python bytecode at any given time.

- **Why it exists:** CPython's memory management is not thread-safe. Without the GIL, concurrent modifications of object reference counts would lead to memory leaks and corruption.
- **Impact:** CPU-bound multi-threaded programs are slow in Python because threads cannot run in parallel on multiple cores.

Q28: Difference between Deep Copy and Shallow Copy.

Answer:

- **Shallow Copy:** Creates a new object but references the nested objects of the original (e.g., list inside a list). Changing a nested object in the copy affects the original.
- **Deep Copy:** Recursively copies the main object and all its nested children. Modifying any part of the deep-copied object leaves the original completely unchanged.

```
import copy
orig = [[1, 2], [3, 4]]
shallow = copy.copy(orig)
deep = copy.deepcopy(orig)

shallow[0][0] = 99 # Modifies orig
deep[1][0] = 999  # Does NOT modify orig
```

Q29: What is the time complexity of a dictionary lookup in Python?

Answer:

- **Average Case:** $O(1)$. Python dictionaries use hash tables, which compute the key's hash and resolve the array index instantly.
- **Worst Case:** $O(n)$. If many keys hash to the same value (hash collisions) and fall into the same bucket, Python must perform linear searches within that bucket. CPython uses open addressing to minimize collisions.

Q30: How does Python manage memory?

Answer:

Memory is managed by the CPython interpreter:

1. **Private Heap:** All Python objects and data structures are stored here. The programmer has no direct access to it.
2. **Reference Counting:** Python tracks the number of references to each object. When an object's reference count drops to 0, its memory is instantly deallocated.
3. **Garbage Collector (GC):** An auxiliary system designed to detect and clean up **reference cycles** (e.g., Class A points to Class B, and Class B points to Class A, preventing reference counts from hitting 0). It uses generational classification (Gen 0, 1, 2) to optimize sweep performance.

Q31: What are the primary types of Joins in SQL?

Answer:

- **INNER JOIN**: Returns records that have matching values in both tables.
- **LEFT (OUTER) JOIN**: Returns all records from the left table, and matching records from the right. If no match, right columns return **NULL**.
- **RIGHT (OUTER) JOIN**: Returns all records from the right table, and matching records from the left.
- **FULL (OUTER) JOIN**: Returns all records when there is a match in either left or right table.
- **CROSS JOIN**: Returns the Cartesian product of both tables (combines every row of the left table with every row of the right).

Q32: Write an example query using an INNER JOIN.

Answer:

```
SELECT
    e.employee_id,
    e.name,
    d.department_name
FROM employees e
INNER JOIN departments d
    ON e.department_id = d.department_id;
```

Q33: Write an example query using a LEFT JOIN.

Answer:

```
SELECT
    e.name,
    p.project_name
FROM employees e
LEFT JOIN projects p
    ON e.employee_id = p.lead_employee_id;
```

Q34: What is the purpose of the GROUP BY clause?

Answer:

The **GROUP BY** clause groups rows that have the same values in specified columns into summary rows. It is almost always used with aggregate functions (like **SUM()**, **AVG()**, **COUNT()**, **MAX()**, **MIN()**) to perform

calculations on each group.

Q35: Explain the difference between HAVING and WHERE.

Answer:

- **WHERE:** Filters records **before** any grouping or aggregation takes place. It operates on individual rows. It cannot contain aggregate functions (e.g., **WHERE SUM(sales) > 100** is invalid).
- **HAVING:** Filters group records **after GROUP BY** has been applied. It operates on aggregate values (e.g., **HAVING SUM(sales) > 1000** is valid).

Q36: What is the difference between a Primary Key and a Foreign Key?

Answer:

- **Primary Key:** Uniquely identifies each record in a table. It must contain unique, non-null values. A table can have only one primary key (which can be a composite key).
- **Foreign Key:** A column (or set of columns) that points to the Primary Key of another table, establishing a referential constraint (relationship) between the tables. It can contain duplicate and NULL values.

Q37: Compare DELETE, TRUNCATE, and DROP statements.

Answer:

- **DELETE:** A Data Manipulation Language (DML) command. Removes rows based on a **WHERE** condition. It is slower because it logs every row deleted. Can be rolled back. Does not reset auto-increment keys.
- **TRUNCATE:** A Data Definition Language (DDL) command. Removes all rows from a table. It is much faster because it deallocates the data pages directly instead of deleting row-by-row. Cannot use **WHERE**. Logs page deallocations. Resets auto-increment keys.
- **DROP:** A Data Definition Language (DDL) command. Completely deletes the entire table structure, indexes, permissions, and data from the database.

Q38: What are SQL Window Functions?

Answer:

Window functions perform calculations across a set of table rows that are related to the current row, without collapsing rows into a single output row. They use the **OVER** clause, which can contain **PARTITION BY** (to group rows) and **ORDER BY** (to order rows within partitions).

- **Example:** Calculating a running total of sales.

```
SELECT
    date, sales,
    SUM(sales) OVER(ORDER BY date) as running_total
FROM daily_sales;
```

Q39: Explain ROW_NUMBER(), RANK(), and DENSE_RANK() with differences.

Answer:

These window functions assign sequential integers to rows within a partition based on an ordering:

- **ROW_NUMBER()**: Assigns unique sequential numbers (1, 2, 3, 4) regardless of ties.
- **RANK()**: Assigns the same rank to tied rows, but skips subsequent numbers (1, 2, 2, 4).
- **DENSE_RANK()**: Assigns the same rank to tied rows, without skipping subsequent numbers (1, 2, 2, 3).

Q40: Write a query to find the second highest salary from an Employee table.

Answer:

Using subqueries:

```
SELECT MAX(salary)
FROM employees
WHERE salary < (SELECT MAX(salary) FROM employees);
```

Using window functions (handles ties properly):

```
WITH RankedSalaries AS (
    SELECT salary,
           DENSE_RANK() OVER (ORDER BY salary DESC) as rnk
    FROM employees
)
SELECT DISTINCT salary
FROM RankedSalaries
WHERE rnk = 2;
```

Q41: Write a query to find and remove duplicate records based on email.

Answer:

Using a Common Table Expression (CTE) with **ROW_NUMBER()**:

```
WITH CTE_Duplicates AS (
    SELECT id, email,
           ROW_NUMBER() OVER (PARTITION BY email ORDER BY id) as row_num
    FROM users
)
DELETE FROM users
WHERE id IN (SELECT id FROM CTE_Duplicates WHERE row_num > 1);
```

Q42: What is the difference between COUNT(*) and COUNT(column)?

Answer:

- **COUNT(*)**: Counts every single row in the table, including rows containing duplicate values and rows containing **NULL** values.
- **COUNT(column)**: Counts only the rows where the specified column is **not NULL**.

Q43: What is a Database Index? Explain Clustered vs Non-Clustered.

Answer:

An index is a database structure that improves the speed of data retrieval operations at the cost of slower writes (INSERT, UPDATE) and additional storage space.

- **Clustered Index**: Determines the physical storage order of the rows in the table. There can be only one clustered index per table (typically created on the Primary Key).
- **Non-Clustered Index**: Creates a separate lookup structure containing the indexed columns and pointers back to the actual data rows. You can have multiple non-clustered indexes.

Q44: Explain Database Normalization.

Answer:

Normalization is the process of organizing database tables to reduce data redundancy and dependency:

- **First Normal Form (1NF)**: All column values must be atomic and each record must be unique.
- **Second Normal Form (2NF)**: Must be in 1NF, and all non-key columns must be fully dependent on the entire primary key (removes partial dependencies).
- **Third Normal Form (3NF)**: Must be in 2NF, and no non-key columns can depend on other non-key columns (removes transitive dependencies).

Q45: What are the ACID properties in databases?

Answer:

- **Atomicity**: The transaction is treated as a single unit; either all operations succeed, or the entire transaction is rolled back (All or Nothing).
- **Consistency**: Ensures the database moves from one valid state to another, maintaining all schema constraints, indexes, and triggers.
- **Isolation**: Concurrent execution of transactions leaves the database in the same state as if they were executed sequentially.
- **Durability**: Once a transaction is committed, its changes are permanently written to non-volatile storage, persisting even during system crashes.

\n# Section 4: Data Structures & Algorithms

Q46: Compare Arrays and Linked Lists.

Answer:

- **Array:** Stored in contiguous memory locations. Offers fast index-based lookup ($O(1)$) but has a fixed size. Insertion/deletion in the middle is slow ($O(n)$) because elements must be shifted.
- **Linked List:** Non-contiguous memory. Nodes contain data and pointers to the next node. Allows dynamic sizing. Insertion/deletion is fast ($O(1)$) if the node pointer is known, but search is slow ($O(n)$) because you must traverse from the head.

Q47: Explain the concepts of Stack and Queue.

Answer:

- **Stack:** A **LIFO** (Last In, First Out) data structure. Elements are added (push) and removed (pop) from the same end (top). Time complexity: $O(1)$ for push and pop.
- **Queue:** A **FIFO** (First In, First Out) data structure. Elements are added (enqueue) at the rear and removed (dequeue) from the front. Time complexity: $O(1)$ for enqueue and dequeue.

Q48: What is a HashMap and how does it handle collisions?

Answer:

A HashMap stores key-value pairs. It uses a hash function to map keys to indexes of an array.

- **Collisions:** Occur when different keys hash to the same array index.
- **Handling Mechanisms:**
 1. *Chaining:* Each index contains a linked list or self-balancing tree of colliding entries.
 2. *Open Addressing:* Finds alternative open slots in the array using linear probing, quadratic probing, or double hashing.

Q49: Write the Binary Search algorithm and state its time complexity.

Answer:

Binary search works by repeatedly dividing a **sorted** search interval in half.

```
def binary_search(arr, target):
    low, high = 0, len(arr) - 1
    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1
```

- **Time Complexity:** $O(\log n)$ because the search space is divided by 2 at each step.
- **Space Complexity:** $O(1)$.

Q50: Compare Merge Sort and Quick Sort.

Answer:

- **Merge Sort:** A divide-and-conquer algorithm. Recursively splits the array, sorts, and merges halves. Time complexity is always $O(n \log n)$. It is a **stable** sort but requires $O(n)$ auxiliary space.
- **Quick Sort:** Selects a pivot and partitions the array. Time complexity is $O(n \log n)$ on average, but $O(n^2)$ in the worst case (if the pivot splits highly unevenly). It is **unstable** but sorts **in-place** ($O(\log n)$ call stack space).

Q51: What is Big O notation? Explain the difference between Time and Space complexity.

Answer:

Big O notation describes the limiting behavior of an algorithm as the input size n grows.

- **Time Complexity:** The execution time of an algorithm expressed in terms of the number of operations it performs.
- **Space Complexity:** The total temporary memory space (auxiliary space + input space) used by the algorithm during execution.

Q52: Solve the "Two Sum" problem in Python with optimal complexity.

Answer:

Using a hash map to achieve $O(n)$ time and $O(n)$ space:

```
def two_sum(nums, target):
    seen = {}
    for i, num in enumerate(nums):
        complement = target - num
        if complement in seen:
            return [seen[complement], i]
        seen[num] = i
    return []
```

Q53: Write a function to reverse a string in-place.

Answer:

Using two pointers:

```
def reverse_string(s: list[str]) -> None:
    left, right = 0, len(s) - 1
    while left < right:
        s[left], s[right] = s[right], s[left]
        left += 1
        right -= 1
```

Q54: Write a function to check if a string is a palindrome.

Answer:

```
def is_palindrome(s: str) -> bool:
    clean = "".join(c.lower() for c in s if c.isalnum())
    left, right = 0, len(clean) - 1
    while left < right:
        if clean[left] != clean[right]:
            return False
        left += 1
        right -= 1
    return True
```

Q55: Find the missing number in an array containing numbers from 1 to N.

Answer:

Using the mathematical sum formula:

```
def find_missing(nums, n):
    expected_sum = n * (n + 1) // 2
    actual_sum = sum(nums)
    return expected_sum - actual_sum
```

Q56: Write a function to detect a cycle in a Linked List.

Answer:

Using Floyd's Cycle Detection Algorithm (Tortoise and Hare):

```
class ListNode:
    def __init__(self, val=0, next=None):
        self.val = val
        self.next = next

def has_cycle(head: ListNode) -> bool:
    slow = fast = head
    while fast and fast.next:
        slow = slow.next
        fast = fast.next.next
        if slow == fast:
            return True
    return False
```

\n# Section 5: Machine Learning Q&A

Q57: Difference between Supervised and Unsupervised Learning.

Answer:

- **Supervised Learning:** The model is trained on labeled training data (features and target). Examples: Predict housing prices (Regression), identify spam email (Classification).
- **Unsupervised Learning:** The model is trained on unlabeled data. It must independently find underlying structural patterns. Examples: Customer segmentation (Clustering), anomaly detection (Autoencoders).

Q58: Explain the Bias-Variance Trade-off.

Answer:

- **Bias:** Error from erroneous assumptions in the model. High bias models are too simple and fail to capture data patterns, leading to **underfitting**.
- **Variance:** Error from sensitivity to small fluctuations in the training set. High variance models capture random noise, leading to **overfitting**.

- **Trade-off:** As model complexity increases, bias decreases but variance increases. The goal is to find the sweet spot that minimizes total error.

Q59: What is Overfitting and how do you prevent it?

Answer:

Overfitting occurs when a model learns the training data and its noise too well, failing to generalize to unseen test data (high training accuracy, low validation accuracy).

- **Prevention:**

1. Use more training data.
2. Simplify the model (reduce features or tree depth).
3. L1/L2 Regularization.
4. Dropout (in neural networks).
5. Early Stopping during training.
6. Ensemble methods (like Random Forest).

Q60: What is Underfitting and how do you prevent it?

Answer:

Underfitting occurs when the model is too simple to learn the underlying structure of the data (low training accuracy, low validation accuracy).

- **Prevention:**

1. Increase model complexity (deeper trees, more layers).
2. Feature Engineering (adding more relevant features).
3. Reduce regularization constraints.
4. Train the model for more epochs.

Q61: What is Cross-Validation? Why is it useful?

Answer:

Cross-validation is a resampling method used to evaluate how a model generalizes. In K-Fold Cross-Validation, the dataset is split into K equal partitions. The model is trained on K-1 partitions and validated on the remaining one. This repeats K times, and scores are averaged.

- **Benefit:** Reduces validation bias because every data point is used for both training and validation.
- **Exception:** Not directly usable on time-series forecasting (where chronological order must be maintained to avoid data leakage). Use TimeSeriesSplit instead.

Q62: What is Feature Engineering? Give examples from your projects.

Answer:

Feature Engineering is the process of using domain knowledge to extract new input features from raw data to boost predictive accuracy.

- **Examples:**

- In my *Retail Forecasting* project: I engineered calendar features (**day_of_week**), shift lags (**lag_7**, **lag_14**), and rolling aggregates (**mean_7**, **std_30**).
- In my *Anomaly Detection* project: I computed Log>Returns on raw stock prices to transform non-stationary price data into a stationary series.

Q63: Explain Precision, Recall, and F1-score.

Answer:

- **Precision:** True Positives (TP) divided by all predicted positives (TP + FP). Out of all records flagged as positive, how many were correct?
- **Recall (Sensitivity):** True Positives (TP) divided by all actual positives (TP + FN). Out of all actual positives, how many did we capture?
- **F1-score:** Harmonic mean of Precision and Recall: $F1 = 2 \times \frac{\text{Precision} \times \text{Recall}}{\text{Precision} + \text{Recall}}$.

Q64: What is a Confusion Matrix?

Answer:

A confusion matrix is an evaluation table summarizing the prediction performance of a classification model:

- **True Positive (TP):** Actual positive predicted as positive.
- **True Negative (TN):** Actual negative predicted as negative.
- **False Positive (FP):** Actual negative predicted as positive.
- **False Negative (FN):** Actual positive predicted as negative.

Q65: Explain the Random Forest algorithm.

Answer:

Random Forest is an ensemble learning method that uses **Bagging** (Bootstrap Aggregating) of Decision Trees:

1. It draws multiple random samples with replacement (bootstrap) from the training set.
2. It trains an independent Decision Tree on each sample. At each split, only a random subset of features is considered.
3. The final prediction is obtained by averaging predictions (Regression) or voting (Classification).

Q66: Explain Gradient Boosting.

Answer:

Gradient Boosting is an ensemble learning method that trains decision trees **sequentially**:

1. A baseline model makes an initial prediction (typically the average target value).
2. The difference between the actual value and prediction (residuals) is calculated.
3. A new weak learner (decision tree) is trained to predict these residuals.
4. The prediction is updated: $\text{New} = \text{Old} + (\text{Learning Rate}) \times \text{Predicted Residual}$.
5. Steps 2-4 repeat, building sequential trees that incrementally reduce the overall error.

Q67: What are the advantages of XGBoost over standard Gradient Boosting?

Answer:

XGBoost (Extreme Gradient Boosting) is an optimized, highly efficient implementation of Gradient Boosting:

- **Regularization:** Applies L1 and L2 regularization to penalize leaf weights, preventing overfitting.
- **Parallel Processing:** Builds tree nodes concurrently by sorting feature thresholds in advance.
- **Hardware Optimization:** Cache-aware data access patterns and out-of-core computing.
- **Missing Values:** Automatically learns default split directions for missing values.

\n# Section 6: Deep Learning Q&A

Q68: What is a Neural Network? How does it learn?

Answer:

A Neural Network is a computational framework consisting of layers of nodes (neurons): Input, Hidden, and Output layers.

- **Learning Process:**

1. *Forward Propagation:* Input features are multiplied by weights, biases are added, and the sum is passed through an activation function to generate predictions.
2. *Loss Calculation:* A loss function measures the error between predictions and ground-truth targets.
3. *Backpropagation:* Gradients of the loss with respect to weights are calculated using the chain rule.
4. *Weight Update:* An optimizer (like SGD or Adam) adjusts weights in the direction that minimizes loss.

Q69: Why do we need Activation Functions in Neural Networks?

Answer:

Activation functions introduce **non-linearity** into the network. Without them, no matter how many layers you stack, the entire network remains a giant linear combination of inputs. Non-linear activations allow networks to learn complex decision boundaries.

Q70: Compare ReLU, Sigmoid, and Softmax activation functions.

Answer:

- **Sigmoid:** $f(x) = 1 / (1 + e^{-x})$. Output ranges between $(0, 1)$. Primarily used in the output layer for binary classification. Disadvantage: Saturates at extreme values, causing **vanishing gradients**.
- **ReLU:** $f(x) = \max(0, x)$. Output ranges from 0 to infinity. It is computationally fast and mitigates vanishing gradients.
- **Softmax:** Converts a vector of raw scores into a probability distribution that sums to 1. Used in the output layer of multi-class classification models.

Q71: Explain Backpropagation.

Answer:

Backpropagation is the method used to calculate the gradient of the loss function with respect to each weight in the neural network. Starting from the output layer, it uses the mathematical **chain rule** of calculus to compute derivatives layer-by-layer backwards. These gradients dictate how weights must be adjusted to reduce error.

Q72: What is Gradient Descent? Compare Batch, Stochastic, and Mini-Batch.

Answer:

Gradient Descent is an optimization algorithm to minimize loss by updating parameters iteratively.

- **Batch:** Calculates gradients using the **entire dataset** before updating. High memory footprint but stable convergence.
- **Stochastic (SGD):** Updates weights after **each single training sample**. Extremely fast updates, but highly volatile path to convergence.
- **Mini-Batch:** Updates weights based on small groups of data (batch sizes of 32, 64, 128). Combines the stability of Batch and the speed of SGD.

Q73: Explain Convolutional Neural Networks (CNNs).

Answer:

CNNs are neural networks designed for grid-like data processing (images). They consist of:

- **Convolutional Layers:** Apply sliding filters (kernels) to extract local spatial features.
- **Pooling Layers:** Downsample feature maps (e.g., Max Pooling) to reduce spatial dimensions and parameter count.
- **Fully Connected Layers:** Flatten feature maps and output classification labels.

Q74: What are Recurrent Neural Networks (RNNs)? What is their primary limitation?

Answer:

RNNs are designed to process sequential data by maintaining a hidden state (memory) that updates at each step.

- **Limitation:** During backpropagation over long sequences, gradients are multiplied repeatedly by weights. If weights are small, gradients shrink exponentially, leading to the **vanishing gradient problem**, causing the network to forget long-term historical context.

Q75: How do LSTMs solve the vanishing gradient problem?

Answer:

LSTMs introduce a **Cell State** (C_t) that acts as a linear conveyor belt running through the sequence, along with three **gating mechanisms** (Forget, Input, Output gates) that use addition rather than continuous multiplication during backpropagation, allowing gradients to flow back through long sequences without vanishing.

\n# Section 7: LLM & Generative AI

Q76: What is a Large Language Model (LLM)?

Answer:

An LLM is a deep learning model containing billions of parameters, typically based on the Transformer architecture. It is pre-trained on massive text datasets (using self-supervised learning) to predict the next token in a sequence. Through this next-token prediction task, it learns grammar, facts, reasoning patterns, and contextual nuances.

Q77: Explain the Transformer architecture.

Answer:

Introduced in 2017, the Transformer replaces recurrent layers with an attention mechanism. It consists of:

- **Encoder:** Processes input text and outputs a sequence of continuous representations.
- **Decoder:** Generates output tokens sequentially by attending to the encoder outputs and previous tokens.
- **Key Components:** Positional encodings, Multi-Head Self-Attention, and Feed-Forward Networks.

Q78: Explain the concept of Self-Attention.

Answer:

Self-Attention allows a model to calculate the relationship between every token in a sequence. For each token, the model computes Query (Q), Key (K), and Value (V) vectors. The dot-product of Query and Key determines the attention weights, which are scaled and passed through a Softmax function, then multiplied by Value.

$$\text{Attention}(Q, K, V) = \text{softmax}\left(\frac{QK^T}{\sqrt{d_k}}\right)V$$

Q79: What are Vector Embeddings?

Answer:

Embeddings are continuous, high-dimensional vector representations of text. They are generated by neural networks such that semantically similar texts are mapped to geometrically close coordinates in vector space.

Q80: What is Tokenization?

Answer:

Tokenization is the preprocessing step of converting raw text strings into discrete integers (tokens) that an LLM can digest. Modern tokenizers use subword algorithms like Byte-Pair Encoding (BPE).

Q81: What is a Context Window?

Answer:

The context window is the maximum number of tokens (input prompts + generated output response) that a Transformer model can process in a single inference call.

Q82: What is Prompt Engineering? Mention common techniques.

Answer:

Prompt Engineering is the practice of designing prompts to guide LLMs toward desired outputs.

- **Techniques:** Zero-Shot, Few-Shot, and Chain-of-Thought (CoT).

Q83: Explain the difference between Temperature and Top-p sampling.

Answer:

- **Temperature:** Scales the raw logits before softmax. Lower temperature makes the distribution peaky (deterministic). Higher temperature flattens logits (creative).
- **Top-p (Nucleus Sampling):** Restricts token choices to the smallest set of tokens whose combined probability exceeds the threshold \$p\$.

Q84: What causes Hallucinations in LLMs?

Answer:

- **Causes:**
 1. *Next-Token Maximization:* The model is optimized for statistical plausibility, not factual accuracy.
 2. *Out-of-Distribution queries:* The model lacks knowledge on specific private data but is forced to generate an output.
 3. *Inaccurate context retrieval* in RAG pipelines.

Q85: What is Retrieval-Augmented Generation (RAG)?

Answer:

RAG is an architecture that extends LLM capabilities by connecting it to external database sources.

- **Workflow:**

1. *Retrieve*: The user's query is embedded, and semantically similar document chunks are fetched from a vector database.
2. *Augment*: These chunks are stuffed into the LLM system prompt as reference context.
3. *Generate*: The LLM uses this retrieved context to write a highly accurate answer.

Q86: What is a Vector Database? Give examples.

Answer:

A vector database is a specialized storage engine designed to store, index, and perform fast similarity searches on high-dimensional vectors. Examples: **FAISS** (library), Chroma, Pinecone.

Q87: What is LangChain?

Answer:

LangChain is an open-source framework designed to simplify the construction of LLM-based applications. It provides standard modules for chains (LCEL), prompts, retrieval structures, and conversation memory.

Q88: Explain Chunking and Chunk Overlap.

Answer:

- **Chunking**: Splitting a long document into smaller, readable segments (e.g., 1000 characters).
- **Chunk Overlap**: The number of characters duplicated between adjacent chunks (e.g., 150 characters) to preserve context.

Q89: What is Semantic Search? How does it differ from Keyword Search?

Answer:

- **Keyword Search**: Matches exact words. If you search "automobile", it might miss "car".
- **Semantic Search**: Computes embeddings of query and text, performing vector search. It captures conceptual meaning.

Q90: What is Cosine Similarity?

Answer:

Cosine similarity measures the similarity of direction between two vectors, ignoring their magnitude.

$$\text{Cosine Similarity}(A, B) = \frac{A \cdot B}{|A| |B|}$$

Section 8: APIs & Deployment

Q91: What is a REST API?

Answer:

Representational State Transfer (REST) is an architectural style for designing networked applications. It relies on stateless client-server communication using HTTP requests (GET, POST, PUT, DELETE).

Q92: Compare GET and POST methods.

Answer:

- **GET:** Requests data. Parameters are sent in the URL query string. Cached by browsers, stored in history, has length limits, and must be idempotent.
- **POST:** Submits data. Parameters are sent in the HTTP request body. Not cached, not saved in history, no size limits, and is non-idempotent.

Q93: Why use FastAPI instead of Flask?

Answer:

- **Asynchronous (async/await):** Native support for concurrent execution.
- **Auto Documentation:** Automatically generates Swagger UI and ReDoc.
- **Type Validation:** Uses Python type hints and Pydantic to automatically validate incoming JSON request payloads.

Q94: What is Docker and why is it useful?

Answer:

Docker packages an application and all its system libraries, runtimes, and dependencies into a single container. It ensures the application runs identically on local laptops, staging, and cloud.

Q95: What is Git and explain a standard branching workflow.

Answer:

Git is a distributed version control system.

- **Workflow:** Developers create feature branches (**feature/add-login**) off **develop**. Once testing is complete, they submit a Pull Request (PR) to merge changes, which are reviewed and merged. Releases are tagged and merged into **main** for production.

Q96: What is CI/CD? Show examples from your projects.

Answer:

- **CI/CD:** Continuous Integration and Continuous Delivery/Deployment.
- In my *Anomaly Detection* project, I configured **Jenkins** to run pytest suites and build a Docker image on new commits.
- In my *Forecasting* project, I wrote **GitHub Actions** workflows to lint and build packages automatically.

Q97: Explain the difference between Azure and AWS cloud environments.

Answer:

Both are leading cloud providers with equivalent services:

- Compute: AWS EC2 matches Azure Virtual Machines.
- Storage: AWS S3 matches Azure Blob Storage / ADLS Gen2.
- Serverless: AWS Lambda matches Azure Functions.
- Big Data: AWS EMR matches Azure Databricks.

\n# Section 9: Project-Based Q&A

Q98: Explain the architecture of your Retail Sales Forecasting project.

Answer:

It is an end-to-end forecasting pipeline built in modular phases:

1. **Data Aggregation:** **data_loader.py** reads raw transaction CSVs and calculates daily metrics.
2. **Feature Engineering:** **feature_engineering.py** adds lags (1, 7, 14, 30 days), rolling aggregates (mean, std), calendar markers, and differences.
3. **Chronological Splitting:** Split data sequentially into 80% train and 20% test (no random shuffling to avoid data leakage).
4. **Model Ensemble:** Trains a Random Forest Regressor and an XGBoost Regressor.
5. **Interactive Dashboard:** A Streamlit dashboard that visualizes forecasts.

Q99: Why did you choose Random Forest and XGBoost? Why not LSTM?

Answer:

- **Random Forest:** Serves as a robust baseline. Being a bagging algorithm, it averages multiple trees, reducing variance.
- **XGBoost:** Selected for high performance. Being a boosting algorithm, it builds trees sequentially to correct preceding errors. Raised R-squared to 0.933.
- **Why not LSTM?:** LSTMs require large amounts of sequential data to converge. For a single store's daily revenue forecast, XGBoost trained in milliseconds and achieved highly accurate results with tabular feature

engineering, bypassing the high compute cost of recurrent networks.

Q100: Explain the details of your RAG Chatbot architecture.

Answer:

The RAG Chatbot is built with a Streamlit UI communicating with a FastAPI backend:

- **Ingestion:** Accepts PDF, DOCX, and TXT files, parsed via LangChain.
- **Chunking:** Splits documents using **RecursiveCharacterTextSplitter** (size=1000, overlap=150).
- **Embeddings:** Generates vectors using HuggingFace **all-MiniLM-L6-v2** wrapped in **CacheBackedEmbeddings** with a disk-based **LocalFileStore**.
- **Vector Index:** FAISS stores vectors on disk.
- **Inference:** User queries are matched against FAISS using MMR retrieval. Context is compiled into a strict anti-hallucination prompt and sent to Groq's Llama-3.3 LLM.

Q101: Why did you choose FAISS and all-MiniLM-L6-v2 embeddings?

Answer:

- **FAISS:** Chosen because it is an open-source, highly efficient library for local vector search. It performs in-memory similarity searches in microseconds and supports disk persistence.
- **all-MiniLM-L6-v2:** A sentence-transformer model that embeds text into 384-dimensional vectors. It represents an excellent balance between search accuracy and light compute footprint.

Q102: What happens if your vector database grows to millions of documents? How would you scale it?

Answer:

1. **Move to a Managed Cluster:** Transition from local FAISS to a managed, distributed vector database like Pinecone, Milvus, or Qdrant.
2. **Index Optimization:** Use quantization strategies like IVF-PQ in FAISS to compress vectors.
3. **Metadata Filtering:** Apply partition filters before performing similarity search.

Q103: Explain SentinelGuard Anomaly Detection System (Project 2).

Answer:

SentinelGuard is an unsupervised system to detect abnormal market movements:

- **Data Processing:** Downloads S&P 500 prices via **yfinance**, calculates log-returns, and builds sequences of length 10.
- **Model:** An **LSTM Autoencoder** trained exclusively on normal, low-variance sequence data.
- **Inference:** Sequences with high reconstruction errors are classified as anomalies.

- **Post-processing:** Implements a 70th percentile noise floor suppression, a rolling 85th percentile dynamic threshold (window=60), and a temporal smoothing filter.

Q104: Explain the training filter used in SentinelGuard.

Answer:

An Autoencoder detects anomalies by learning a compressed representation of "normal" behavior. If anomalous data is present during training, the autoencoder will learn to reconstruct anomalies, resulting in low reconstruction errors on future anomalies.

- **Solution:** I implemented a two-step training filter:
 1. Filter out all sequences containing known historical anomaly labels.
 2. Compute the variance of each sequence and discard the top 20% highest-variance sequences, training purely on calm, low-volatility baseline sequences.

Q105: Detail the LSTM Autoencoder configuration in SentinelGuard.

Answer:

```
# Encoder
inputs = Input(shape=(10, 1))
x = LSTM(64, return_sequences=True)(inputs)
x = LSTM(32, return_sequences=False)(x)
bottleneck = Dense(16, activation="tanh")(x)

# Decoder
x = RepeatVector(10)(bottleneck)
x = LSTM(32, return_sequences=True)(x)
x = LSTM(64, return_sequences=True)(x)
outputs = TimeDistributed(Dense(1))(x)
```

Q106: Explain the YOLO object detection project. Why YOLO instead of Faster R-CNN?

Answer:

- **YOLO (You Only Look Once):** A single-stage object detector. It processes the entire image in a single forward pass, predicting bounding boxes and class probabilities simultaneously, making it extremely fast (30+ FPS) and suitable for real-time applications.
- **Faster R-CNN:** A two-stage detector (region proposals + classification). While slightly more accurate, it is computationally heavy and too slow for real-time edge deployment. YOLO was selected to guarantee high-FPS real-time inference.

\n# Section 10: AWS Cloud Services Q&A

Q107: Explain AWS EC2 and instance families.

Answer:

AWS Elastic Compute Cloud (EC2) provides resizable virtual servers. Instance families are optimized for different workloads:

- **General Purpose (T, M):** Balanced compute, memory, and networking.
- **Compute Optimized (C):** High-performance processors for batch processing and ML inference.
- **Memory Optimized (R, X):** High RAM for in-memory databases and PySpark processing.
- **Accelerated Computing (G, P):** GPU-equipped instances for deep learning model training.

Q108: What is AWS S3? Explain its storage classes.

Answer:

Simple Storage Service (S3) is an object storage service.

• Storage Classes:

1. *Standard*: Frequently accessed data, low latency.
2. *Intelligent-Tiering*: Automatically moves data between tiers based on access patterns.
3. *Standard-IA*: Lower storage cost but retrieval fees apply. Good for backups.
4. *S3 Glacier*: Lowest cost archiving.

Q109: Compare AWS RDS and DynamoDB.

Answer:

- **RDS (Relational Database Service):** Managed relational database engine supporting PostgreSQL, MySQL, SQL Server. Uses structured schemas and enforces full ACID compliance.
- **DynamoDB:** Managed NoSQL key-value database. Provides single-digit millisecond latency at any scale. Fully serverless and scales horizontally.

Q110: What is AWS IAM? Explain Users, Groups, Roles, and Policies.

Answer:

Identity and Access Management (IAM) controls access:

- **User:** Individual identity with credentials.
- **Group:** Collection of users.
- **Role:** An identity with no credentials, assumed temporarily by users or services.
- **Policy:** A JSON document defining permissions.

Q111: What is AWS Lambda?

Answer:

AWS Lambda is a serverless, event-driven compute service. You upload code and Lambda runs it only when triggered by events (e.g., S3 file uploads, API Gateway requests). You pay only for the compute time consumed in milliseconds.

Q112: Compare AWS Glue and AWS EMR for Spark workloads.

Answer:

- **AWS Glue:** Serverless ETL tool. It automatically discovers, catalogs metadata, and runs PySpark jobs without requiring server management.
- **AWS EMR (Elastic MapReduce):** Managed cluster platform that allows fine-grained control over Apache Spark, Hadoop, and Hive nodes.

Q113: What is AWS SageMaker?

Answer:

SageMaker is an end-to-end managed service for machine learning:

- **Notebooks:** Managed Jupyter environments.
- **Training:** Spins up transient GPU clusters, copies training data from S3, trains the model, saves artifacts, and terminates.
- **Endpoints:** Deploys models as autoscaling REST APIs.

Q114: Compare CloudWatch and CloudTrail.

Answer:

- **CloudWatch:** Monitors performance, metrics, and application logs. Used to track resource utilization and alarms.
- **CloudTrail:** Records API calls and user actions across your AWS account for security auditing.

Q115: What is a VPC? Explain Public vs Private Subnets.

Answer:

A Virtual Private Cloud (VPC) is a logically isolated virtual network inside AWS.

- **Public Subnet:** Connected to an Internet Gateway. Resources have public IP addresses and can receive direct inbound traffic from the internet.
- **Private Subnet:** No direct route to the internet. Resources have private IPs. They can access the internet outbound only via a NAT Gateway located in the public subnet.

Q116: What is AWS Redshift?

Answer:

Redshift is a managed, petabyte-scale data warehouse service. It uses columnar storage and Massively Parallel Processing (MPP) to execute analytical queries (OLAP) on large datasets extremely quickly.

\n# Section 11: PySpark & Big Data Q&A

Q117: What is Apache Spark and PySpark?

Answer:

- **Apache Spark:** An open-source, distributed general-purpose cluster-computing framework that processes massive datasets in-memory.
- **PySpark:** The Python API for Apache Spark, allowing developers to write Spark applications using Python code.

Q118: Explain Spark's Master-Worker Architecture.

Answer:

- **Driver Program:** Runs the main application code, compiles it into a DAG, and schedules execution.
- **Cluster Manager:** Allocates cluster resources.
- **Executors (Workers):** Distributed JVM processes on worker nodes that execute tasks, process partitions, and store data in-memory or on disk.

Q119: Compare RDDs, DataFrames, and Datasets in Spark.

Answer:

- **RDD (Resilient Distributed Dataset):** Spark's basic low-level abstraction. A distributed collection of JVM objects. Lacks optimization.
- **DataFrame:** A distributed table structured with named columns. Heavily optimized using the Catalyst Optimizer.
- **Dataset:** A strongly-typed extension of DataFrames available in Scala/Java, but not supported in PySpark.

Q120: What is Lazy Evaluation in Spark? Compare Transformations and Actions.

Answer:

Spark does not execute computations immediately. It builds a logical execution plan (DAG) of the steps to execute.

- **Transformations:** Operations that create a new DataFrame from an existing one (e.g., `filter()`, `join()`). They are lazy.
- **Actions:** Operations that return a value to the driver or write output to storage (e.g., `count()`, `show()`). They trigger physical execution.

Q121: What is the Spark Web UI? How do you use it for debugging?

Answer:

The Spark Web UI is a browser dashboard exposed on port 4040. It provides detailed metrics about Jobs, Stages, Tasks, and memory storage. It is used to locate slow stages, identify data skew, and monitor memory consumption.

Q122: What is the difference between Partitioning and Bucketing?

Answer:

- **Partitioning:** Splits data into physical subdirectories on disk based on a partition column. Best for low-cardinality columns.
- **Bucketing:** Distributes data into a fixed number of files (buckets) based on the hash value of a column. Best for high-cardinality columns frequently joined with other tables.

Q123: What is Shuffling in Spark? How do you avoid it?

Answer:

Shuffling is the process of redistributing data across executors over the network, triggered by wide transformations. It is the most expensive operation in Spark.

- **Avoidance:**
 1. Use **Broadcast Joins** when joining a large table with a small table.
 2. Bucket both tables on the join key.
 3. Apply filters early.

Q124: Write a PySpark snippet to read a CSV and run a GROUP BY aggregation.

Answer:

```
from pyspark.sql import SparkSession
from pyspark.sql.functions import col, sum, avg

spark = SparkSession.builder.appName("SalesAnalysis").getOrCreate()
df = spark.read.csv("sales.csv", header=True, inferSchema=True)

result_df = df.filter(col("quantity") > 0) \
    .groupBy("category") \
    .agg(
        sum("revenue").alias("total_revenue"),
        avg("price").alias("average_price")
    )
result_df.show()
```

Q125: What is Data Skew and how do you handle it in PySpark?

Answer:

Data skew occurs when one or a few partitions receive significantly more data than others, slowing down the entire stage.

- **Handling:**

1. *Salting*: Appending a random prefix to the join key to distribute the skewed data.
2. *Filtering Nulls*: Filter out null join keys.
3. *Broadcasting*: If one table is small, broadcasting it bypasses the partitioning shuffle.

Q126: What is Delta Lake and what are its key features?

Answer:

Delta Lake is an open-source storage layer that runs on top of Parquet files to provide reliability to data lakes:

- **ACID Transactions**: Guarantees that data operations either fully succeed or roll back.
- **Time Travel**: Maintains a transaction log, allowing users to query older snapshots.
- **Schema Enforcement**: Prevents corrupt or malformed data writes.
- **Unified Batch/Streaming**: Supports writing and reading streaming and batch data.